



GIT

Big Data H/M 2022

Richard McCreadie

WHAT IS GIT?

- Git is **version control system**
 - This means it is a remote storage system that can hold versioned copies of your files
 - It is important to use a version control system so you have a second copy of your code, as hardware failures and human error do happen
- Version control systems differ from generic remote storage services like Google Drive, Microsoft's OneDrive and Apple iCloud, in that they don't just store the most recent copy of the data, but also **intermediate versions**
 - This means there is a 'paper-trail' of changes showing **who did what and when**
 - It means if you make a major mistake you can **roll-back** to an earlier version

CREATING A NEW PROJECT

 **GitLab** Menu



Create blank project

Create a blank project to house your files, plan your work, and collaborate on code, among other things.

New project > Create blank project

Project name

My awesome project

Project URL

https://stgit.dcs.gla.ac.uk/

big-data-m-h/2021/athena

Project slug

my-awesome-project

Want to house several dependent projects under the same namespace? [Create a group.](#)

Project description (optional)

Description format

Visibility Level 

☒ Private

Project access must be granted explicitly to each user. If this project is part of a group, access will be granted to members of the group.

☒ Initialize repository with a README

Allows you to immediately clone this project's repository. Skip this if you plan to push up an existing repository.

Create project

Cancel

CLONE

- Now that we have created a remote project where we can store stuff, we now want to be able to access it locally
- This is done with `git clone`
 - Creates a local copy of a remote project

The screenshot shows a GitHub repository page for a project named 'test' (Project ID: 3893). The repository has 1 commit, 1 branch, 0 tags, 143 KB of files, and 143 KB of storage. The 'main' branch is selected, and the 'test' directory is shown. The repository contains an 'Initial commit' by Richard McCreddie, authored just now. The repository also has a README, Auto DevOps enabled, and options to add a LICENSE, CHANGELOG, or additional files. A 'Clone' button is visible in the top right corner. A dropdown menu is open, showing options to clone the repository using SSH or HTTPS, or to open it in an IDE (Visual Studio Code).

Clone with SSH

```
git@stgit.dcs.gla.ac.uk:bigdata-m
```

Clone with HTTPS

```
https://stgit.dcs.gla.ac.uk/bigda
```

Open in your IDE

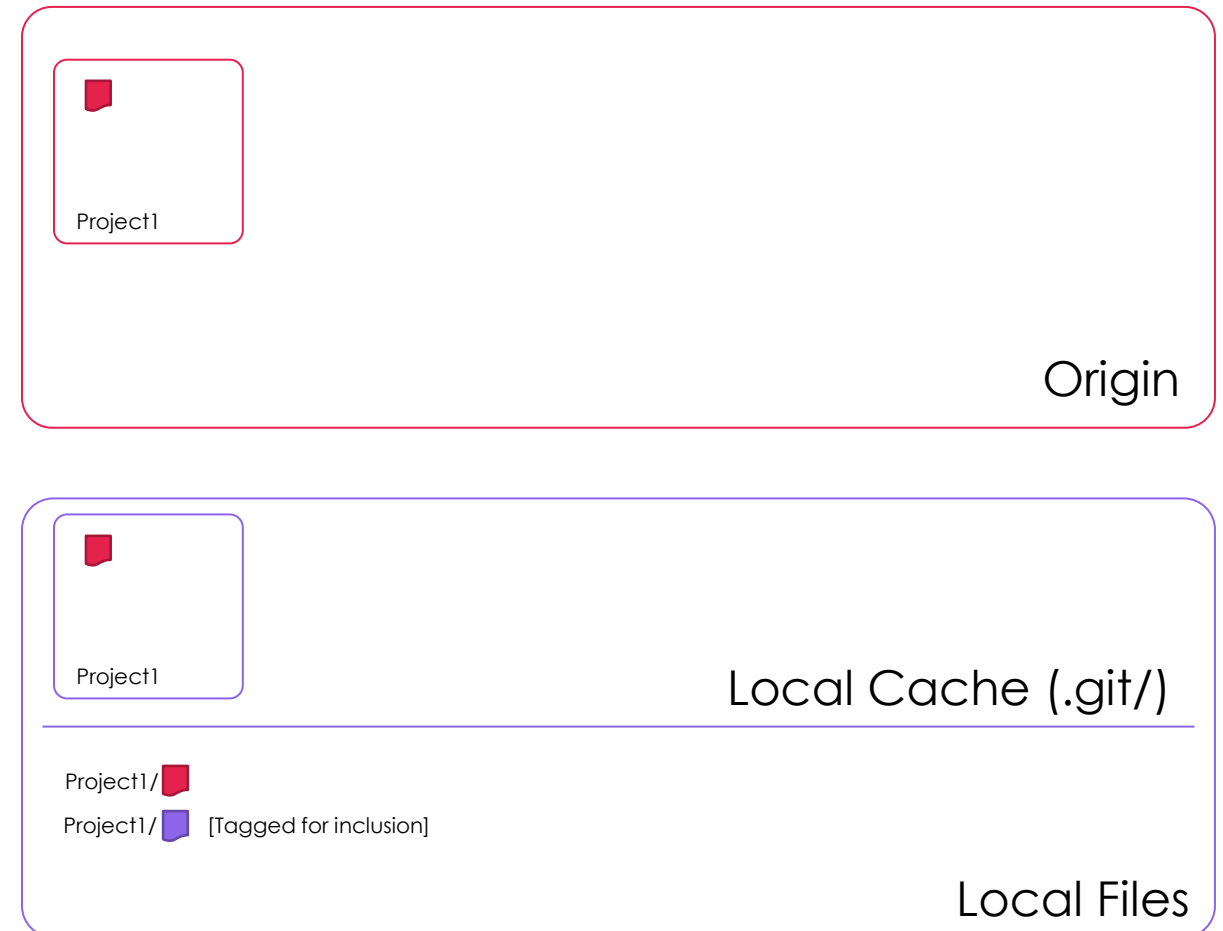
- Visual Studio Code (SSH)
- Visual Studio Code (HTTPS)

Copy URL

Name	Last commit
README.md	Initial commit

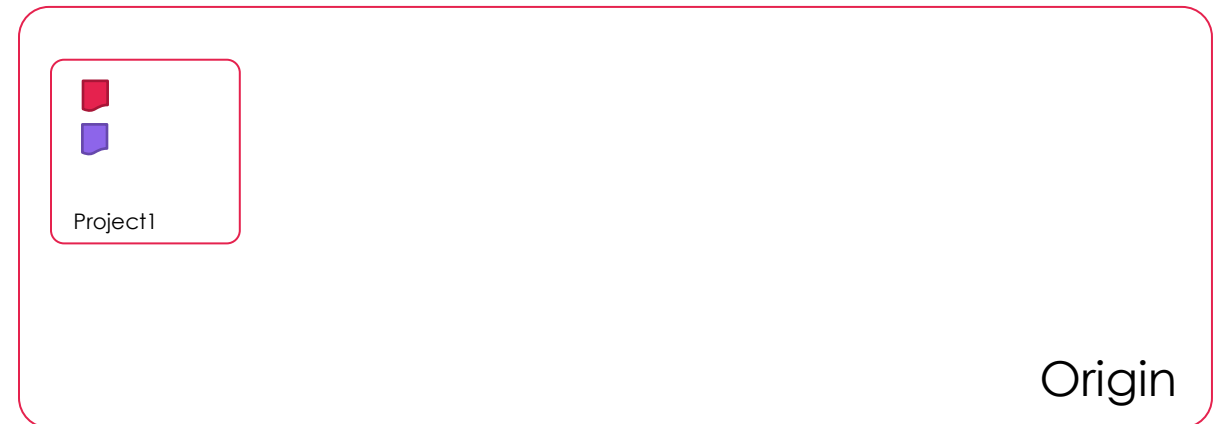
LOCAL VS REMOTE

- Cloning copies the origin files to both the local cache and updates the local files
- Lets create a new file locally
 - This file is initially not tracked by git
- We use the `git add` command to tell git we want to track it in our project



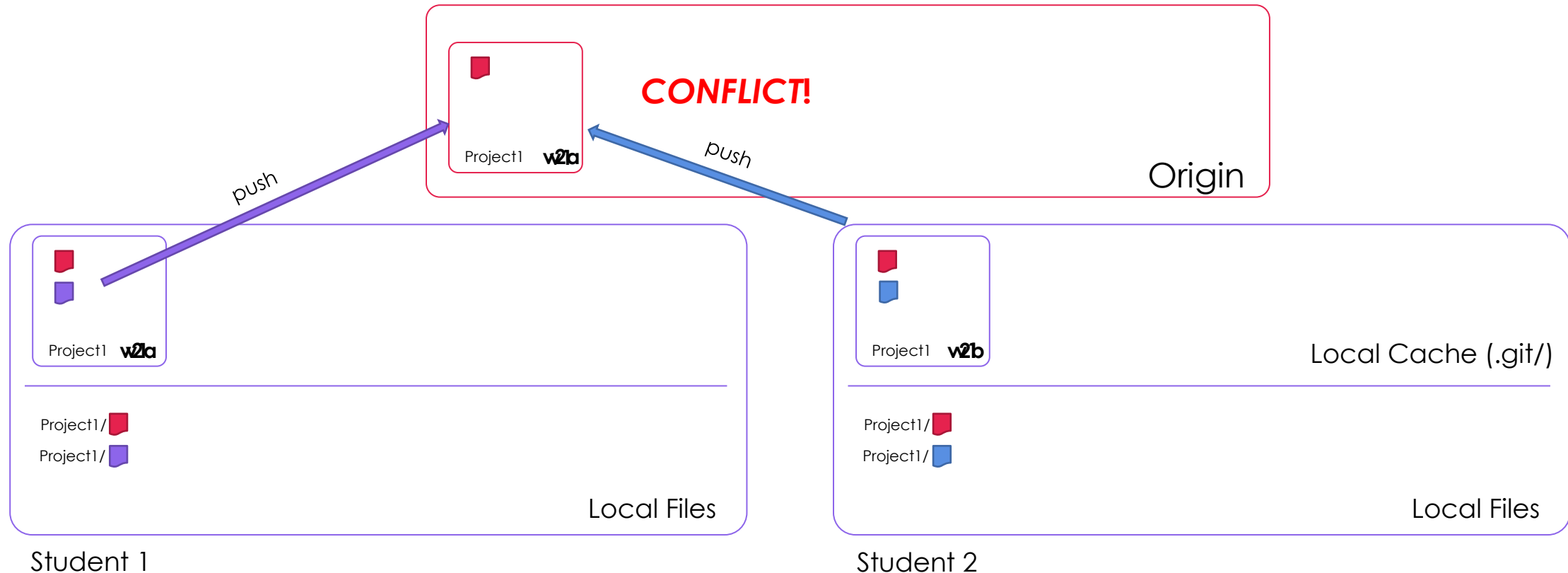
LOCAL VS REMOTE

- To add it to our local cache we need to issue a commit command
 - `git commit -m "message"`
- Committing transfers changes from your local files to your local cache
- To update the remote repository we need to push our local changes
 - `git push`



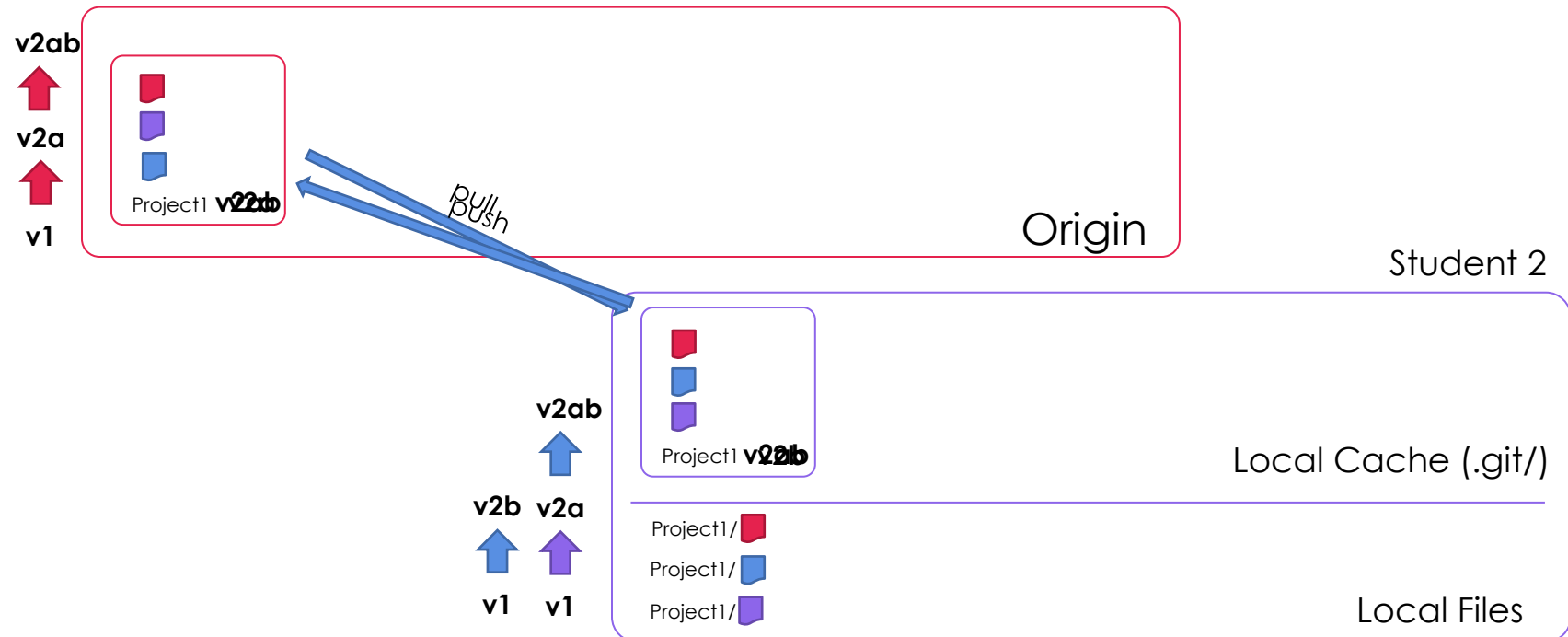
CONFLICTS

- You will have multiple people working on your exercise, and as such, multiple people will end up committing to a single project, which can cause **conflicts**



LINEAR COMMITS

- Commits to the remote repository in git can only happen in time order
 - If a local copy is 'behind' in terms of commits to the remote, the local version will need to merge the remote's changes (becoming up-to-date) before its own changes will be accepted





MERGING

- To merge remote changes with your local cache you need to call
 - `git pull`
- At this point two things might occur
 - 1) the files in your local commit do not overlap with the changes in the remote, allowing git to perform the merge automatically
 - 2) one or more files you have committed have also changed in the remote, meaning you need to resolve the conflict manually
 - In this case, git will edit your local copy

TYPES OF FILES THAT CAN BE STORED

- Git can store two main types of files, although its only optimised for one of these, **text** files and **binaries**
- For text files, git is quite efficient, in that it will only store the difference between versions
 - This saves significant amounts of space, and makes it easier to see what changed across versions
- For binary files (i.e. everything else), it just stores a new copy of the file for each version
 - This is one of the most common reasons for git repositories running out of space